

PROTOCOL

Technical Whitepaper

Proof of Useful Work (PoUW) Consensus Protocol in the zk-Tensor Core Architecture

Abstract

This document specifies the technical architecture of the PROTOCOL ecosystem. It defines the mathematical foundations of the Proof of Useful Work (PoUW) consensus, built on an optimized linear-algebra engine (the zk-Tensor Core). The system implements error-free arithmetic decomposition into INT8 format, cryptographic tensor fragmentation (Matrix Tiling), exact precision aggregation, and commitment-bound probabilistic verification via the Stochastic Matrix Audit (SMA) algorithm. The architecture is complemented by a scalable data-availability layer with transparent polynomial commitments (P-DAL + FRI) and a private, post-quantum transaction layer (zk-TWL).

Contents

1	System Architecture and Base Parameters	2
1.1	Ecosystem Parameters Table	2
1.2	Supply Function (Smooth Emission)	2
1.3	Internal Fee Market	2
2	Computational Layer: zk-Tensor Core	3
2.1	Generalized Tensor Equation (GTE) and Zero-Padding	3
2.2	Error-Free Decomposition and Cryptographic Tensor Fragmentation	3
2.3	Precision Aggregation Protocol (Result Rollup)	5
2.4	Anti-Fraud Mechanism: Result Binding and Sequential Time Seal	5
3	Consensus, Penalties and Economics (PoUW)	6
3.1	Sybil Resistance and Quarantine	6
3.2	Automated Proportional Settlements (Pay-Per-Share)	6
3.3	Dual-Mode Architecture and Difficulty Regulator (PID)	7
4	Transaction and Data Storage Layer (P-DAL)	7
4.1	Data Availability Layer (P-DAL) and FRI Commitments	7
4.2	Post-Quantum Transactional Model (zk-TWL)	8
4.3	Block Structure and State Integration	8

1 System Architecture and Base Parameters

PROTOCOL relies on a deterministic monetary policy. Token emission follows a deflationary curve, which stabilizes the profitability of the computational infrastructure while maintaining a fixed maximum supply.

1.1 Ecosystem Parameters Table

Component	Technical Description
Maximum Supply ($S_{\max}$)	Asymptotically approaches 15,000,000 [TKN].
Genesis Allocation	2,250,000 [TKN] allocated in Block 0, intended for development, incentive mechanisms, and the testnet.
Emission Model	10% annual decrease ($\rho = 0.10$).
Algorithmic Consensus	Proof of Useful Work (PoUW) optimized for linear algebra, based on INT8 arithmetic with exact floating-point reconstruction and the SMA audit.
Entry Barrier and Penalties	No capital stake required (0 TKN). Sybil resistance derives from the cost of verified work (Pay-Per-Share model): every identity must independently perform and pass validation of its own shares. Penalties rely on block rejection and Quarantine.
Transaction Privacy	zk-TWL (zk-STARK proofs and CRYSTALS-Dilithium signatures) preserving full transfer confidentiality.

Table 1: Ecosystem Parameters.

1.2 Supply Function (Smooth Emission)

Emission reduction is modeled as a geometric progression. The total supply in year t is:

$$S_{\text{current}}(t) = G + \sum_{n=0}^t \bar{R}_0 (1 - \rho)^n = G + \bar{R}_0 \frac{1 - (1 - \rho)^{t+1}}{\rho} \quad (1)$$

The asymptotic limit defines the fixed maximum supply:

$$S_{\max} = \lim_{t \rightarrow \infty} S_{\text{current}}(t) = G + \frac{R_0}{\rho} = 2,250,000 + \frac{1,275,000}{0.10} = 15,000,000 \text{ [TKN]} \quad (2)$$

Legend:

- G : Genesis Allocation generated in Block 0 (2,250,000 [TKN]).
- $S_{\text{current}}(t)$: Integrated token supply in the network in year t .
- R_0 : Base reward in the first year (Genesis Epoch), 1,275,000 [TKN].
- ρ : Annual emission reduction rate, 0.10.
- t : Year index since network initialization.

1.3 Internal Fee Market

The internal pricing of a computational unit is dynamically calibrated by market mechanisms:

$$P_{\text{compute}} = \frac{Fee_{\text{task}}}{N_{\text{ops}}} \quad (3)$$

Legend:

- P_{compute} : Profit per elementary computational unit, in [TKN].
- Fee_{task} : Total task budget, in [TKN].
- N_{ops} : Estimated volume of computational operations (a function of matrix size).

2 Computational Layer: zk-Tensor Core

The network engine executes mathematical operations using the zk-Tensor Core architecture, optimized for large-scale tasks.

2.1 Generalized Tensor Equation (GTE) and Zero-Padding

Every problem introduced to the network is reduced to a Generalized Tensor Equation (GTE):

$$A \times B + \Omega = C \quad (4)$$

The engine operates over a configurable algebraic domain: over a finite ring for cryptographic tasks ($A \times B + \Omega \equiv C \pmod{p^k}$), or over floating-point numbers for numerical tasks, where product exactness is guaranteed by the error-free decomposition (Sec. 2.2).

For asymmetric or irregular matrices, the network automatically applies the Zero-Padding technique, virtually expanding the data edges. This allows lossless task division without affecting the final mathematical result.

Application		Matrix A	Matrix B	Noise/Shift Ω	Result C
Artificial Intelligence	Intelli-	Network Weights (W)	Input Data (X)	Bias Vector	Activations
Fluid Mechanics		Stiffness Matrix (K)	System State (U)	Force Vector (F)	Balance
Lattice Cryptography	Cryptogra-	Lattice Basis (A)	Secret Key (S)	Gaussian Noise (E)	Public Key

Table 2: Implementations of the Generalized Tensor Equation.

2.2 Error-Free Decomposition and Cryptographic Tensor Fragmentation

To let researchers maintain high floating-point precision without burdening the silicon, the network implements an error-free arithmetic decomposition. Instead of statistical noise averaging, each floating-point matrix is decomposed into a sum of bounded-width integer slices whose products are computed exactly in INT8 arithmetic with INT32 accumulation.

Slice decomposition: For a matrix $A \in \mathbb{R}^{M \times K}$ we select a slice width of b bits and decompose:

$$A = \sum_{i=1}^{s_A} 2^{-\sigma_i^A} A^{(i)}, \quad A^{(i)} \in \mathbb{Z}^{M \times K}, \quad |A_{kl}^{(i)}| < 2^b \quad (5)$$

where σ_i^A is the scale exponent of slice i , and the number of slices $s_A = \lceil \mu/b \rceil$ depends on the requested mantissa precision μ (in bits). Dynamic range is handled by per-tile scaling. Matrix B is decomposed analogously.

Exact product: By linearity, the full product is reconstructed from exact integer partial products:

$$AB = \sum_{i=1}^{s_A} \sum_{j=1}^{s_B} 2^{-(\sigma_i^A + \sigma_j^B)} A^{(i)} B^{(j)} \quad (6)$$

Each partial product $A^{(i)}B^{(j)}$ is computed on the tensor core in INT8 $\rightarrow$ INT32 format with no rounding. The no-overflow condition for the accumulator, for a tile of inner dimension n , is:

$$2b + \lceil \log_2 n \rceil \leq 31 \quad (7)$$

For INT8 operands ($b \leq 7$) and $n = 512$ we obtain $14 + 9 = 23 \leq 31$, with margin. Slice pairs whose combined scale falls below the target unit in the last place (ULP) are pruned, which reduces the number of products to a triangular subset of size $\approx s_A s_B / 2$.

The no-overflow bound is a checkable invariant with headroom. The condition above is a protocol invariant, verified per configuration rather than assumed. At $b = 7$ it admits an inner tile dimension up to $n = 2^{31-14} = 2^{17}$ before INT32 saturates, so the reference parameters ($b = 7, n = 512$) sit far inside the safe region. Should a configuration ever approach the limit, the bound is restored either by finer slicing (a smaller b , which increases the slice count s) or by widening the accumulator to INT64; the choice is an efficiency trade-off, not a correctness one.

Target	Precision	Mantissa Bits μ	Slices $s = \lceil \mu/b \rceil$ ($b = 7$)	INT8 Products (after pruning)
FP16		11	2	order of units
FP32		24	4	order of 10
FP64		53	8	order of tens

Table 3: Reconstruction cost: a fixed, deterministic quantity. For comparison, a noise-averaging approach would require on the order of 10^6 iterations for FP32 and 10^{24} for FP64.

Cryptographic Tensor Fragmentation (Matrix Tiling): To eliminate the hardware I/O bottleneck (the "Memory Wall"), gigantic input matrices $A \in \mathbb{R}^{M \times K}$ and $B \in \mathbb{R}^{K \times N}$ are partitioned into standardized sub-matrices (tiles) of fixed block dimension $n \times n$ (e.g. $n = 512$). Formally, the inputs are represented as block structures:

$$A = [A_{i,k}]_{i=1,k=1}^{M',K'}, \quad B = [B_{k,j}]_{k=1,j=1}^{K',N'} \quad (8)$$

where $M' = \lceil M/n \rceil$, $K' = \lceil K/n \rceil$, and $N' = \lceil N/n \rceil$. A miner processes exclusively its assigned tile, producing a local sub-result $C_{i,j|k}$ (the tile fragment i, j originating from block k). That sub-result is itself the exact scaled sum of the bounded integer partial products $A_{i,k}^{(\cdot)} B_{k,j}^{(\cdot)}$ of the slice decomposition.

Integrity Audit (SMA): A claimed matrix product $XY = Z$ is verified deterministically in $\mathcal{O}(n^2)$ time rather than the $\mathcal{O}(n^3)$ cost of re-multiplication by drawing a challenge vector $\vec{h} \in \mathbb{F}_p^n$ over a finite field (its derivation and binding to the result are given in Sec. 2.4) and checking the identity:

$$X(Y\vec{h}) - Z\vec{h} = \vec{0} \pmod{p} \quad (9)$$

By the associativity of matrix multiplication the column vector $Y\vec{h}$ is evaluated first, which reduces the total cost of validation to $\mathcal{O}(n^2)$ and allows on-the-fly computation within the ultra-fast L1/L2 cache of the GPU. In PROTOCOL the audit is applied to each bounded integer partial product of the decomposition, i.e. $X = A^{(i)}$, $Y = B^{(j)}$, $Z = A^{(i)}B^{(j)}$.

Integer-exactness of the audit (no modular wraparound): Because the SMA identity is checked on the bounded integer partial products $A^{(i)}B^{(j)}$, every audited entry obeys the no-overflow bound and is therefore strictly smaller than the field modulus:

$$|[A^{(i)}B^{(j)}]_{kl}| < 2^{2b+\lceil \log_2 n \rceil} \leq 2^{31} \ll p = 2^{61} - 1 \quad (10)$$

Consequently, reduction modulo p acts as the identity on the audited quantities: a passing audit certifies exact equality over $\mathbb{Z}$, not merely congruence modulo p . The classical gap in which an

adversary submits $C' = AB \pmod{p} \neq AB$ and passes with probability 1 cannot arise here, because the audited values never reach p . The full, potentially large product is reconstructed only by exact summation in FP64, outside the field (Sec. 2.3), and is never itself reduced modulo p . An explicit range assertion on the revealed entries (cost $\mathcal{O}(n^2)$) may accompany the audit to make this integer-equivalence verifiable by inspection.

Soundness: If $A^{(i)}B^{(j)} \neq Z$, then for $\vec{h}$ drawn uniformly from $\mathbb{F}_p^n$:

$$\Pr[(A^{(i)}B^{(j)} - Z)\vec{h} = \vec{0}] \leq \frac{1}{p} \quad (11)$$

A single challenge over $p = 2^{61} - 1$ thus bounds the cheating probability by $\approx 2^{-61}$; equivalently, an adversary grinding candidate results would need $\approx 2^{61}$ attempts below a comfortable cryptographic margin. The protocol therefore amplifies soundness by drawing r independent challenge vectors from the same commitment (Sec. 2.4):

$$\vec{h}^{(l)} = H(\kappa \parallel \text{Block}_{\text{prev}} \parallel l) \in \mathbb{F}_p^n, \quad l = 1, \dots, r \quad (12)$$

which yields a bound of p^{-r} . With $r = 3$ this is $\approx 2^{-183}$ (grinding cost $\approx 2^{183}$, infeasible); $r = 2$ is already $\approx 2^{-122}$. We adopt $r = 3$ as the default, exceeding a 128-bit security margin with headroom. Each field element is obtained from the hash output by rejection sampling (or from a wider digest) to remove the slight non-uniformity of reducing a 64-bit value modulo $2^{61} - 1$; this bias is cryptographically negligible but is eliminated for cleanliness.

2.3 Precision Aggregation Protocol (Result Rollup)

When the network has validated and stored all component partial products, the aggregation layer reconstructs the floating-point result by an exact summation of the verified integer partial products with their corresponding scales:

$$C_{\text{float}} = \sum_{(i,j) \in P} 2^{-(\sigma_i^A + \sigma_j^B)} C_{\text{int}}^{(i,j)} \quad (13)$$

$$C_{\text{int}}^{(i,j)} = A^{(i)}B^{(j)} \quad (14)$$

Legend:

- P : The set of retained slice pairs (after pruning terms below the ULP).
- $C_{\text{int}}^{(i,j)}$: The exact INT32 partial product.
- σ_i^A, σ_j^B : The scale exponents of the slices.

The summation is performed in FP64. The result is exact down to the target ULP, with no statistical error and no dependence on a number of repetitions. The client receives a ready, precise result, maintaining full abstraction of the protocol's hardware operations.

2.4 Anti-Fraud Mechanism: Result Binding and Sequential Time Seal

To secure the system against pre-computation attacks and identifier manipulation (grinding), the audit uses a commit-reveal scheme (the Fiat-Shamir heuristic), and block finalization uses a Verifiable Delay Function (VDF).

Result-to-challenge binding procedure:

1. Commitment: the miner publishes $\kappa = H(C \parallel \text{TaskID} \parallel \text{MinerID})$ before the challenge is known.

2. Challenge derivation: the r challenge vectors are derived deterministically as $\vec{h}^{(l)} = H(\kappa \parallel \text{Block}_{\text{prev}} \parallel l) \in \mathbb{F}_p^n$ for $l = 1, \dots, r$.
3. Reveal and verification: the miner reveals C ; the node checks $H(C \parallel \dots) = \kappa$ and the r SMA equations.

Because each $\vec{h}^{(l)}$ is a deterministic function of the commitment to C , an adversary cannot construct $C \neq AB$ satisfying $(AB - C)\vec{h}^{(l)} = \vec{0}$; any change to C changes κ , and therefore every $\vec{h}^{(l)}$. This closes adaptive forgery, and grinding a passing commitment requires simultaneously defeating all r independent challenges, i.e. probability p^{-r} .

Sequential Time Seal (VDF):

$$(y, \pi) = VDF(x, T) \quad (15)$$

$$x = H(\kappa \parallel \text{Block}_{\text{prev}}) \quad (16)$$

The VDF requires T sequential (inherently non-parallelizable) steps and produces a proof π verifiable in $\mathcal{O}(\text{polylog } T)$ time. A block is valid only with a correct pair (y, π) . The mechanism enforces a minimal, non-parallelizable time interval and removes the advantage of pre-computation and identifier grinding.

Note on post-quantum security: Classic VDF constructions (repeated squaring in groups of unknown order) are not post-quantum. If strict post-quantum resistance is required, the delay function can be realized as a sequential hash chain (hash-based assumptions), at the cost of a less succinct proof. Manipulation of timing alone is of limited value compared to forging computation, which is independently prevented by the binding scheme.

3 Consensus, Penalties and Economics (PoUW)

3.1 Sybil Resistance and Quarantine

Resistance to Sybil attacks derives from the cost of verified work, not from a hardware identity. In the Pay-Per-Share model, reward is proportional to validated work; creating multiple identities yields no advantage, because each one must independently perform the computation and pass validation of its own shares. The barrier is the real cost of computation and energy.

The MinerID is a cryptographic identifier derived from the node's public key. Reputation is an asset earned through correctly validated work and may gate access to higher-value tasks; Quarantine causes its forfeiture together with any unsettled rewards.

3.2 Automated Proportional Settlements (Pay-Per-Share)

Breaking massive tasks into tiles transforms profit distribution into a proportional model that pays for the actual amount of executed and validated work:

$$W_{\text{node}} = \left(\frac{v_{\text{tiles}}}{V_{\text{total}}} \right) \cdot \text{Fee}_{\text{task}} [\text{TKN}] + R_{\text{block}}(t) \quad (17)$$

Legend:

- W_{node} : Total node revenue in a given settlement window.
- v_{tiles} : Number of tiles correctly validated in the block by the miner.
- V_{total} : Total volume of tiles in the client's task.
- Fee_{task} : [TKN] Market premium declared by the client for the full model.

Incident Type	Protocol Procedure	Consequence
Computational Error / Failure	Failure to meet the mathematical conditions of the SMA algorithm for the processed tile.	Share/block rejection. The node loses the energy (electricity) used for generation, with no reward allocation.
Forgery Attempt	Detection of result manipulation by the binding scheme (Sec. 2.4) or on-chain data forgery.	Activation of Quarantine (Timeout Jail). Forfeiture of unsettled rewards and of the accumulated reputation associated with the identifier.

Table 4: Security and Penalty Policy.

- $R_{\text{block}}(t)$: Protocol emission reward for supporting the network in epoch t .

3.3 Dual-Mode Architecture and Difficulty Regulator (PID)

The system operates in two modes, distinguished by a binary switch on the availability of paid tasks.

Commercial Mode: Difficulty is exogenous, imposed by the client’s real task. The protocol does not regulate it; it only prices it through the fee market (Sec. 1.3).

Synthetic Mode: Activated when the paid task pool is empty. Difficulty becomes endogenous and is steered by a PID regulator that maintains a target block time:

$$D(n) = D_{\text{base}} + K_p e(n) + K_i \sum_{k=0}^n e(k) + K_d [e(n) - e(n-1)] \quad (18)$$

Legend:

- $e(n)$: $e(n) = \tau_{\text{target}} - \tau_{\text{actual}}(n)$ is the block-time error at step n .
- K_p, K_i, K_d : The proportional, integral, and derivative gains.

The integral term eliminates steady-state error; the derivative term damps oscillations. In Synthetic Mode, the protocol allocates available capacity to public-good tasks: optimization of open models (Open Weights) and generation of cryptographic infrastructure (ZK proofs). This maintains the utility of the network outside the context of commercial tasks.

4 Transaction and Data Storage Layer (P-DAL)

4.1 Data Availability Layer (P-DAL) and FRI Commitments

The data-availability layer manages the transfer and storage of tensor structures, relying on probabilistic Data Availability Sampling (DAS). Heavy input tiles of matrices $A_{i,k}$ and $B_{k,j}$ undergo linear serialization into elements over a finite field $\mathbb{F}_p$, creating a global block data vector $D = (d_0, d_1, \dots, d_{m-1})$.

Reed-Solomon error-correction coding: To make the network resilient against Data-Withholding attacks, the data vector D is interpolated as a unique polynomial $\varphi(X) \in \mathbb{F}_p[X]$ of degree $\deg(\varphi) < m$ satisfying $\varphi(\alpha_l) = d_l$ for a fixed set of base points $\{\alpha_l\}_{l=0}^{m-1}$. This polynomial is extended using a Reed-Solomon code with redundancy factor $R = 0.5$ by evaluating it over an extended domain $D_{\text{ext}} \subset \mathbb{F}_p$ of size $2m$:

$$D_{\text{ext}} = (\varphi(\beta_0), \varphi(\beta_1), \dots, \varphi(\beta_{2m-1})) \quad (19)$$

FRI polynomial commitment: The integrity of the encoded structure is represented in the block header through a Merkle root over the evaluations on the extended domain:

$$C_{P-DAL} = \text{Merkle Root}(\varphi(\beta_0), \varphi(\beta_1), \dots, \varphi(\beta_{2m-1})) \quad (20)$$

The commitment is hash-based (post-quantum) and requires no trusted setup.

Sample verification using DAS: During sampling, light nodes download random samples $(\beta_z, \varphi(\beta_z))$ together with a Merkle authentication path and verify it against the root, which binds the sample value. The correctness of the encoding (i.e. that the data is the evaluation of a polynomial of degree $< m$) is attested by a FRI low-degree proof π_{FRI} , operating through a logarithmic number of folding rounds with Fiat-Shamir challenges. The node checks two conditions:

1. $\text{MerklePath}(\beta_z, \varphi(\beta_z)) \rightarrow C_{P-DAL}$
2. $\text{VerifyFRI}(\pi_{FRI}, C_{P-DAL})$

The cost of verifying a single sample is $\mathcal{O}(\text{polylog } m)$, and soundness is amplified by the number of random queries. Resistance to Data-Withholding follows from the Reed-Solomon redundancy: the availability of a sufficient fraction of correct samples enables reconstruction of the full block.

Note on the trade-off: FRI openings are larger than the constant-size KZG openings (polylogarithmic versus constant). In return, the construction gains post-quantum resistance, transparency (no SRS / trusted setup), and consistency with the zk-STARK layer.

4.2 Post-Quantum Transactional Model (zk-TWL)

Transaction validity rests on the value-conservation axiom:

$$\sum_{i=1}^m I_i = \sum_{j=1}^n O_j + Fee \quad (21)$$

PROTOCOL implements a Dual-Pool zk-TWL system supported by post-quantum cryptography:

- **Shielded Pool:** Abandons vulnerable elliptic curves. Uses zk-STARK proofs for absolute privacy. Cryptographic commitments are computed inside the circuit using collision-resistant hash functions ($C = \text{Hash}(v \parallel r)$). Operations are authorized using post-quantum CRYSTALS-Dilithium (ML-DSA) signatures. Nullifiers prevent double-spending.
- **Transparent Pool:** Used exclusively to publish computational contracts and lock rewards (Fee_{task}), giving the network dynamic service pricing without compromising privacy.

4.3 Block Structure and State Integration

The main ledger evaluates only verification descriptors:

$$CID = \text{SHA-256}(\text{MatrixA} \parallel \text{MatrixB}) \quad (22)$$

The block header integrates the cryptographic and financial state:

$$Header = \text{Hash}(\text{PrevHash} \parallel \text{Root}_{TWL} \parallel \text{Root}_{PoUW} \parallel \text{MinerID} \parallel \text{VDF}_{out} \parallel \text{Timestamp}) \quad (23)$$

Legend:

- CID: Content Identifier in the availability layer.
- MatrixA, MatrixB: Component data of the client's task.

- Header: Full cryptographic hash of the validated block header.
- PrevHash: Reference descriptor of the previous block.
- RootTWL: Merkle root for monetary transfers (zk-TWL pool).
- RootPoUW: Merkle root for references to scientific matrices and SMA evaluations (under the binding scheme).
- MinerID: Cryptographic identifier of the block producer.
- VDF_{out} : Output of the Verifiable Delay Function (time anchor).
- Timestamp: Deterministic timestamp of block closure.